GEMPRO Notebook

GEMPRO

Contest

template.cpp

Template base para abrir um problema e começar a implementar.

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); i++)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

Comandos de shell para compilar, rodar e comparar saídas rápido.

```
c() { g++ $1.cpp -o $1 -std=c++20; }
cs() {
    g++ $1.cpp -o $1 -std=c++20 \
        -fsanitize=address,undefined \
        -fsanitize=signed-integer-overflow -ggdb
}
gen() {
    local N=${1:-10}
    for i in $(seq 1 $N); do
        python3 gen.py $i > $2$i.in
    done
}
rr() {
    local ext=${2:-out}
    for i in $(ls -v *.in); do
        local f=$(basename $i .in)
        ./$1 < $f.in > $f.$ext
    done
}
chk() {
    local a=${1:-ans}
    local b=${2:-out}
    for i in $(ls -v *.$a); do
        local f=$(basename $i .$a)
        echo -n "$f.$a - $f.$b > "
        if cmp -s $f.$a $f.$b; then
            echo OK
        else
            echo X
        fi
    done
}
```

Template de stress testing

Template de gerador.

```
from random import randrange as rr, shuffle as sh, sample as
↪  sm, choice as ch, seed
from string import ascii_lowercase as alpha
from itertools import combinations as combs, permutations as
↪  perms, product as prod
from sys import argv

if len(argv) > 1: seed(argv[1])
```

.vimrc

Configuração do Vim para contest.

```
set cin ts=4 sw=4 udf cul nu is
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
↪  md5sum \| cut -c -6
```

Matemática

Algoritmo Euclideano

`g = egcd(a, b, x, y)`, $ax + by = g$. Complexidade: $O(\log(\min(a, b)))$.

b4f64d - math/egcd.cpp - 9 lines

```cpp
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

Raiz quadrada módulo `mod`

`sqrtMod(a)` retorna a menor raiz de x^2 `%` $mod = a$, ou `-1`. $O(\log^2 mod)$.

b04743 - math/mod-sqrt.cpp - 19 lines

```cpp
int sqrtMod(int a) { // mod prime
    if (a == 0 || mod == 2) return a;
    if (powm(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return powm(a, (mod + 1) / 4);
    int s = __builtin_ctz(mod - 1), q = (mod - 1) >> s, z = 2;
    while (powm(z, (mod - 1) / 2) != mod - 1) z++;
    ll c = powm(z, q), x = powm(a, (q + 1) / 2), t = powm(a,
    ↪  q);
    for (int m = s; t != 1; ) {
        ll tt = t;
        int i = 0;
        while (tt != 1) tt = tt * tt % mod, i++;
        ll b = powm(c, 1 << (m - i - 1));
        x = x * b % mod;
        c = b * b % mod;
        t = t * c % mod;
        m = i;
    }
    return min<ll>(x, mod - x);
} // b04743
```

Soma de Floor

Somas com piso: `floorSum`, `floorSumI` e `floorSumSq`; custo logarítmico.

84ad33 - math/floor-sum.cpp - 45 lines

```cpp
using i128 = __int128_t;

i128 floorSumSq(ll n, ll m, ll a, ll b);

i128 floorSum(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * n * (n - 1) / 2 + (i128)qb * n
            + floorSum(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * (n - 1) - floorSum(k, a, m, m - b - 1);
} // 440661

i128 floorSumI(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) *
    ↪  (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * s2 + (i128)qb * s1
            + floorSumI(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * s1
        - (floorSumSq(k, a, m, m - b - 1) + floorSum(k, a, m,
        ↪  m - b - 1)) / 2;
} // dd026c
i128 floorSumSq(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) *
    ↪  (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * qa * s2 + 2 * (i128)qa * qb * s1 +
        ↪  (i128)qb * qb * n
            + 2 * (i128)qa * floorSumI(n, m, a % m, b % m)
            + 2 * (i128)qb * floorSum(n, m, a % m, b % m)
            + floorSumSq(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * k * (n - 1)
        - 2 * floorSumI(k, a, m, m - b - 1)
        - floorSum(k, a, m, m - b - 1);
} // eb1940
```

CRT

`crt(r1, m1, r2, m2)` combina duas congruências em tempo logarítmico.

6e0e1c - math/crt.cpp - 9 lines

```cpp
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Base de XOR

`add`, `minXor` e `maxXor`; $O(B)$ por operação.

0e002d - math/xor-basis.cpp - 20 lines

```cpp
struct XorBasis {
    static const int B = 63; // works for nonnegative ll
    ↪  values < 2^63
    ll b[B] = {};
    bool add(ll x) {
        for (int i = B - 1; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) return b[i] = x, 1;
            x ^= b[i];
        }
        return 0;
    } // 9dfd7d

    ll minXor(ll x) {
        for (int i = B - 1; i >= 0; i--) x = min(x, x ^ b[i]);
        return x;
    } // f6abc2
    ll maxXor(ll x = 0) {
        for (int i = B - 1; i >= 0; i--) x = max(x, x ^ b[i]);
        return x;
    } // e00cc1
};
```

Teste de primalidade

`isPrime(n)` para inteiros de 64 bits em $O(\log n)$.

b5a468 - math/primality.cpp - 25 lines

```cpp
using i128 = __int128;

bool checkPrime(ll p, ll a) {
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p == 1) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    ↪  {
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // a137c8
```

Fatoração de inteiros

`factor(n, fs)` adiciona em `fs` os fatores primos de `n`; usa `isPrime`.

7b1297 - math/factor.cpp - 28 lines

```cpp
mt19937_64 rng64(random_device{}());

using ull = unsigned long long;
using u128 = __uint128_t;

ull modMul(ull a, ull b, ull mod) { return (u128)a * b % mod;
↪  }

ull rho(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ull c = uniform_int_distribution<ull>(1, n - 1)(rng64);
```

```
    ull x = uniform_int_distribution<ull>(0, n - 1)(rng64), y
    ↪  = x, d = 1;
    auto f = [&](ull x) { return (modMul(x, x, n) + c) % n; };
    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? rho(n) : d;
} // 8e1254

void factor(ull n, vector<ull> &fs) {
    if (n == 1) return;
    if (isPrime(n)) { fs.pb(n); return; }
    ull d = rho(n);
    factor(d, fs);
    factor(n / d, fs);
} // 6d58e9
```

Interpolação de Lagrange

`lagrange(xs, ys, x)` e `lagrange(ys, x)` calculam o valor do polinômio de Lagrange no ponto `x`. A segunda função assume que os valores de `y` são dados para `x = 0, 1, ..., n - 1`. Complexidade de `lagrange(xs, ys, x)`: $O(n^2 \log(mod))$. Complexidade de `lagrange(ys, x)`: $O(n)$.

29e498 - math/lagrangian-interpolation.cpp - 26 lines

```
ll lagrange(const vector<ll> &xs, const vector<ll> &ys, ll x)
↪  {
    int n = sz(xs); ll ans = 0;
    rep(i,0,n) if (xs[i] == x) return ys[i];
    rep(i,0,n) {
        ll a = ys[i], b = 1;
        rep(j,0,n) if (i != j) a = a * (x - xs[j] + mod) %
        ↪  mod, b = b * (xs[i] - xs[j] + mod) % mod;
        ans = (ans + a * powm(b, mod - 2)) % mod;
    }
    return ans;
} // 51ae24

ll lagrange(const vector<ll> &y, ll x) {
    int n = sz(y);
    x = (x % mod + mod) % mod;
    if (x < n) return y[x];
    vector<ll> p(n + 1, 1), s(n + 1, 1), a(n);
    rep(i,0,n) p[i + 1] = p[i] * (x - i + mod) % mod;
    for (int i = n - 1; i >= 0; --i) s[i] = s[i + 1] * (x - i
    ↪  + mod) % mod;
    a[0] = 1;
    rep(i,1,n) a[i] = a[i - 1] * i % mod;
    a[n - 1] = powm(a[n - 1], mod - 2);
    for (int i = n - 1; i; --i) a[i - 1] = a[i] * i % mod;
    ll ans = 0;
    rep(i,0,n) ans = (ans + p[i] * s[i + 1] % mod * a[i] % mod
    ↪  * a[n - 1 - i] % mod * (((n - 1 - i) & 1) ? mod - y[i]
    ↪  : y[i])) % mod;
    return ans;
} // 05868c
```

Polinômio de Lagrange

`lagrangePoly(xs, ys)` retorna o polinômio que passa pelos pontos (x_i, y_i) para `i = 0, 1, ..., n - 1`. Complexidade: $O(n^2)$.

7d0710 - math/lagrangian-polynomial.cpp - 19 lines

```
vector<ll> lagrangePoly(const vector<ll> &xs, const vector<ll>
↪  &ys) {
    int n = sz(xs);
    vector<ll> p = {1}, f(n);
    rep(i,0,n) {
        p.pb(0);
        for (int j = i + 1; j; --j) p[j] = (p[j - 1] + mod -
        ↪  p[j] * xs[i] % mod) % mod;
        p[0] = p[0] * (mod - xs[i]) % mod;
    }
    rep(i,0,n) {
        vector<ll> q(n);
        q[n - 1] = p[n];
        for (int j = n - 1; j; --j) q[j - 1] = (p[j] + q[j] *
        ↪  xs[i]) % mod;
        ll den = 1;
        rep(j,0,n) if (i != j) den = den * (xs[i] - xs[j] +
        ↪  mod) % mod;
        ll c = ys[i] * powm(den, mod - 2) % mod;
        rep(j,0,n) f[j] = (f[j] + c * q[j]) % mod;
    }
    return f;
} // 7d0710
```

Berlekamp-Massey

Encontra os coeficientes da recorrência que forma `s`. Complexidade: $O(n^2)$.

68a616 - math/berlekamp-massey.cpp - 19 lines

```
vi berlekampMassey(vi s) {
    vi c(1, 1), b(1, 1);
    int l = 0, m = 1; ll bb = 1;
    rep (n, 0, sz(s)) {
        ll d = 0;
        rep (i, 0, l + 1) d = (d + 1LL * c[i] * s[n - i]) %
        ↪  mod;
        if (d < 0) d += mod;
        if (!d) { m++; continue; }
        vi t = c;
        ll x = d * powm(bb, mod - 2) % mod;
        if (sz(c) < sz(b) + m) c.resize(sz(b) + m);
        rep (i, 0, sz(b)) c[i + m] = (c[i + m] - x * b[i]) %
        ↪  mod;
        if (2 * l <= n) l = n + 1 - l, b = t, bb = d, m = 1;
        else m++;
    }
    c.erase(begin(c));
    for (int &x : c) x = (mod - x) % mod;
    return c;
} // 68a616
```

Recorrência linear

Encontra o k-ésimo termo da recorrência linear pelos primeiros termos e os coeficientes. Complexidade: $O(n^2 \log(k))$.

5f7ef4 - math/linear-recurrence.cpp - 23 lines

```
int linRec(vi s, vi tr, ll k) { // s = first m terms,
↪  0-indexed kth term
    int n = sz(tr);
    if (k < n) return s[k];
    auto mul = [&](vi a, vi b) {
        vi c(2 * n + 1);
        rep (i, 0, n + 1) rep (j, 0, n + 1)
            c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep (j, 0, n)
            c[i - 1 - j] = (c[i - 1 - j] + 1LL * c[i] * tr[j])
            ↪  % mod;
        c.resize(n + 1);
        return c;
    };
    vi pol(n + 1), e(n + 1);
    pol[0] = 1;
    e[1] = 1;
    for (++k; k; k >>= 1) {
        if (k & 1) pol = mul(pol, e);
        e = mul(e, e);
    }
    ll ans = 0;
    rep (i, 0, n) ans = (ans + 1LL * pol[i + 1] * s[i]) % mod;
```

```
    return ans;
} // 5f7ef4
```

Eliminação Gaussiana

Realiza eliminação gaussiana na matriz. Complexidade: $O(n^3)$.

3843ca - math/gaussian-elimination.cpp - 18 lines

```
int gauss(vector<vector<ll>> &a) {
    int n = sz(a), m = sz(a[0]), r = 0;
    rep(c,0,m) {
        int p = r;
        while (p < n && !a[p][c]) p++;
        if (p == n) continue;
        swap(a[p], a[r]);
        ll iv = powm((a[r][c] % mod + mod) % mod, mod - 2);
        rep(j,c,m) a[r][j] = a[r][j] * iv % mod;
        rep(i,0,n) if (i != r && a[i][c]) {
            ll x = a[i][c];
            rep(j,c,m) a[i][j] = (a[i][j] - x * a[r][j]) %
            ↪ mod;
        }
        r++;
    }
    rep(i,0,n) rep(j,0,m) if (a[i][j] < 0) a[i][j] += mod;
    return r;
} // 3843ca
```

Permutação para inteiro

Converte permutação para inteiro e vice-versa.

df96dc - math/perm-int.cpp - 20 lines

```
ll perm2int(vi p) {
    vi a(sz(p)); iota(all(a), 0); ll r = 0;
    for (int x: p) {
        int i = find(all(a), x) - a.begin();
        r = r * sz(a) + i;
        a.erase(a.begin() + i);
    }
    return r;
} // 01ca9e

vi int2perm(int n, ll k) {
    vi a(n), p; iota(all(a), 0); ll f = 1;
    rep(i,1,n) f *= i;
    rep(i,0,n) {
        int j = k / f; k %= f;
        p.pb(a[j]); a.erase(a.begin() + j);
        if (n - i - 1) f /= n - i - 1;
    }
    return p;
} // edd4ac
```

Estruturas de Dados

Segtree

Segtree iterativa genérica: `set(i, x)` e `query(l, r)` em $O(\log n)$.

b76e87 - dsa/segtree.cpp - 18 lines

```
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i <<
        ↪ 1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy Segtree

Lazy segtree genérica: `set`, `apply` e `sum` em $O(\log n)$.

21bbc4 - dsa/lazy-segtree.cpp - 50 lines

```
template<class S, S (*op)(S, S), S (*e)(), class U, S
↪ (*mapping)(U, S),
        U (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(4 * N, id()), n(N) {}
    void applyNode(int i, U u) {
        t[i] = mapping(u, t[i]);
        d[i] = compose(u, d[i]);
    } // 3e5d0d
    void push(int i) {
        applyNode(i << 1, d[i]);
        applyNode(i << 1 | 1, d[i]);
        d[i] = id();
    } // 7836a8
    void pull(int i) { t[i] = op(t[i << 1], t[i << 1 | 1]); }
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        if (tl + 1 == tr) {
            t[i] = val;
            d[i] = id();
            return;
        }
        push(i);
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
        else set(j, val, tm, tr, i << 1 | 1);
        pull(i);
    } // f9c2fd
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) return applyNode(i, u);
        push(i);
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
        pull(i);
    } // 3f0fb7
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
    S sum(int l, int r, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
        push(i);
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1),
                  sum(l, r, tm, tr, i << 1 | 1));
    } // b976f6
};
```

Sparse Table

Estrutura estática para idempotentes: build $O(n \log n)$ e query $O(1)$.

2e4ed0 - dsa/sparse-table.cpp - 18 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
```

```
    vi lg;
    void build(vector<S> &v) {
        int n = ssize(v);
        lg.assign(n + 1, 0);
        rep (i, 2, n + 1) lg[i] = lg[i >> 1] + 1;
        t.assign(lg[n] + 1, vector<S>(n));
        t[0] = v;
        rep (k, 1, lg[n] + 1)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k - 1][i], t[k - 1][i + (1 <<
                ↪  (k - 1))]);
    } // 9c11fe
    S query(int l, int r) {
        int k = lg[r - l];
        return op(t[k][l], t[k][r - (1 << k)]);
    } // d2d23e
};
```

Mo Queries

Retorna a ordem das queries no algoritmo de Mo. Complexidade das queries: $O(N\sqrt{(Q)})$.

c2c90b - dsa/mo-queries.cpp - 11 lines

```
vi moOrd(vector<pii> &q, int n) {
    int B = max(1, (int)sqrt(n));
    vi o(sz(q));
    iota(all(o), 0);
    sort(all(o), [&](int i, int j) {
        int a = q[i].fi / B, b = q[j].fi / B;
        if (a != b) return a < b;
        return (a & 1) ? q[i].se > q[j].se : q[i].se <
        ↪  q[j].se;
    });
    return o;
} // c2c90b
```

Convolução

FFT e NTT para multiplicar polinômios em $O(n \log n)$.

FFT

c8b36c - dsa/fft.cpp - 38 lines

```
using cd = complex<ld>;
const ld PI = acosl(-1.0L);

void fft(vector<cd> &a) {
    int n = sz(a), lg = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        rt.resize(n);
        cd x = polar((ld)1, PI / k);
        rep(i, k, 2 * k) rt[i] = i & 1 ? rt[i / 2] * x : rt[i
        ↪  / 2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | ((i & 1) << lg)) / 2;
    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            rep(j, 0, k) {
                cd z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // c30ab4

vector<ld> conv(vector<ld> &a, vector<ld> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    vector<cd> in(n), out(n);
    rep(i, 0, sz(a)) in[i].real(a[i]);
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (cd &x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    vector<ld> res(s);
    rep(i, 0, s) res[i] = imag(out[i]) / (4 * n);
    return res;
} // 6d98ab
```

NTT

7c4fe0 - dsa/ntt.cpp - 40 lines

```
// const int mod = 998244353, ROOT = 62;
void ntt(vector<ll> &a) {
    int n = sz(a);
    static array<ll, 1 << 23> r = {1, 1};
    for (static int t = 2, s = 2; t < n; t <<= 1, s++) {
        array z = {1ll, powm(ROOT, mod >> s)};
        for (int i = t; i < t << 1; i++) r[i] = r[i >> 1] *
        ↪  z[i & 1] % mod;
    }
    static array<ll, 1 << 23> b, c;
    copy(all(a), b.begin());
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(b[i], b[j]);
    }
    for (int m = 1; m < n; m <<= 1) {
        for (int l = 0; l < n; l += m << 1)
            rep (i, 0, m) {
                ll z = r[m + i] * b[l + m + i] % mod;
                c[l + i] = b[l + i] + z;
                c[l + m + i] = b[l + i] + (mod - z);
            }
        rep (i, 0, n) b[i] = c[i] < mod ? c[i] : c[i] - mod;
    }
    rep(i, 0, n) a[i] = b[i];
} // c057a3

vector<ll> conv(vector<ll> &a, vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    int inv = powm(n, mod - 2);
    vector<ll> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n) out[-i & (n - 1)] = L[i] * R[i] % mod * inv %
    ↪  mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // ad22f1
```

Convolução de XOR

`xorConv(a, b)` para convolução por XOR em $O(n \log n)$.

fd3fd7 - dsa/xor-convolution.cpp - 20 lines

```
void fwht(vector<ll> &a) {
    for (int k = 1; k < sz(a); k <<= 1)
        for (int i = 0; i < sz(a); i += 2 * k)
            rep (j, i, i + k) {
                ll x = a[j], y = a[j + k];
                a[j] = x + y; // mod: a[j] = (x + y) % mod;
                a[j + k] = x - y; // mod: a[j + k] = (x - y +
                ↪  mod) % mod;
            }
} // ae23e1

vector<ll> xorConv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < max(sz(a), sz(b))) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
```

```
    rep (i, 0, n) a[i] *= b[i]; // mod: a[i] = a[i] * b[i] %
    ↪ mod;
    fwht(a);
    rep (i, 0, n) a[i] /= n; // mod: a[i] = a[i] * invN %
    ↪ mod;
    return a;
} // 0bc2c7
```

Zeta e Mobius

Transformadas em subconjuntos para vetores indexados por máscara, $O(B2^B)$.

34c15b - dsa/zeta.cpp - 68 lines

```
vi subsetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep (i, 1, n)
            if (i & j) a[i] += a[i ^ j];
    return a;
} // b39c8d
vi subsetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i] -= a[i ^ j];
    return a;
} // 5ffc86
vi supersetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] += a[i];
    return a;
} // c5d556
vi supersetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] -= a[i];
    return a;
} // 128136
vi enumPrimes(int n) {
    vi pr, spf(n);
    rep(i, 2, n) {
        if (spf[i] == 0) spf[i] = i, pr.pb(i);
        for (int j: pr) {
            if (i * j >= n) break;
            spf[i * j] = j;
            if (j == spf[i]) break;
        }
    }
    return pr;
} // 46cddf
vi divZeta(vi a) {
    int n = sz(a) - 1;
    for (int p: enumPrimes(n + 1))
        for (int i = 1; i * p < n; i++)
            a[i * p] += a[i];
    return a;
} // 320d42
vi divMobius(vi a) {
    int n = sz(a) - 1;
    for (int p: enumPrimes(n + 1))
        for (int i = n / p; i; i--)
            a[i * p] -= a[i];
    return a;
} // b5c104
vi mulZeta(vi a) {
    int n = sz(a) - 1;
    for (int p: enumPrimes(n + 1))
        for (int i = n / p; i; i--)
            a[i] += a[i * p];
    return a;
} // be1f11
vi mulMobius(vi a) {
    int n = sz(a) - 1;
    for (int p: enumPrimes(n + 1))
        for (int i = 1; i * p < n; i++)
            a[i] -= a[i * p];
    return a;
} // 8d0ceb
```

Convolução de subconjuntos

Convolução de OR para subconjuntos disjuntos. Complexidade: $O(n \log(n)^2)$.

0b06ec - dsa/subset-convolution.cpp - 16 lines

```
#define T popcount(unsigned(i))
vi subsetConv(vi &a, vi &b) {
    int n = sz(a), m = bit_width(unsigned(n));
    vector ca(m, vi(n)), cb(m, vi(n)), c(m, vi(n));
    rep(i, 0, n) ca[T][i] = a[i], cb[T][i] = b[i];
    rep(i, 0, m) ca[i] = subsetZeta(ca[i]), cb[i] =
    ↪ subsetZeta(cb[i]);
    rep(i, 0, n) rep(j, 0, m) rep(k, 0, m - j) {
        c[j + k][i] += ll(ca[j][i]) * cb[k][i] % mod;
        if (c[j + k][i] >= mod) c[j + k][i] -= mod;
    }
    rep(i, 0, m) c[i] = subsetMobius(c[i]);
    vi r(n);
    rep(i, 0, n) r[i] = c[T][i];
    return r;
} // 916ee1
#undef T
```

Árvore de Li Chao

Envelope de retas para mínimo: `insert` e `query` em $O(\log N)$.

9e88be - dsa/li-chao.cpp - 29 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} //
    ↪ Replace with LLONG_MIN in case of finding max
    void insert(Line line) { insert(line, 0, n, 1); }
    void insert(Line line, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace
        ↪ with > in case of finding max
        if (tl + 1 == tr) return;
        if (t[i](tl) < line(tl)) insert(line, tm, tr, i << 1 |
        ↪ 1); // Replace with > in case of finding max
        else insert(line, tl, tm, i << 1);
    } // f784a4
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN
        ↪ in case of finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with >
        ↪ in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore de Li Chao++

Li Chao com retas ativas só em subintervalos, $O(\log^2 N)$.

e28ce0 - dsa/li-chao-extended.cpp - 36 lines

```cpp
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} //
    ↪ Replace with LLONG_MIN in case of finding max
    void insert(Line line, int L, int R) { insert(line, L, R,
    ↪ 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int
    ↪ i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]); //
            ↪ Replace with > in case of finding max
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm,
            ↪ tr, i << 1 | 1); // Replace with > in case of
            ↪ finding max
            else insert(line, L, R, tl, tm, i << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 3dcc38
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN
        ↪ in case of finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with >
        ↪ in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Calculando todas as somas possíveis para subconjuntos

Subset sum com complexidade $O(\frac{nS}{64})$.

a4f267 - dsa/fast-subset-sum.cpp - 22 lines

```cpp
vi subSum(vi x) {
    using ull = unsigned long long;
    int s = reduce(all(x));
    vector<ull> bs(s / 64 + 1);
    bs[0] = 1;
    vi p(s + 1, -1);
    rep(i,0,sz(x)) {
        int a = x[i], q = a / 64, r = a % 64;
        for (int j = sz(bs) - 1; j >= 0; --j) {
            ull pre = bs[j], add = 0;
            if (j >= q) add |= (q ? bs[j - q] : pre) << r;
            if (r && j > q) add |= bs[j - q - 1] >> (64 - r);
            bs[j] |= add;
            for (ull k = bs[j] ^ pre; k; k &= k - 1) {
                int t = __builtin_ctzll(k);
                int v = 64 * j + t;
                if (v <= s) p[v] = i;
            }
        }
    }
    return p;
} // a4f267
```

Árvore Cartesiana

`cartTree(n, le)` retorna o pai de cada índice em $O(n)$.

caeaba - dsa/cartesian-tree.cpp - 20 lines

```cpp
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.pb(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // caeaba
```

Treap implícita

Treap de sequência com `split`/`merge`; custo esperado $O(\log n)$.

ad716f - dsa/treap.cpp - 69 lines

```cpp
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};

int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }

void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b),
    ↪ pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
```

```
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488

void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Simplex

Maximiza $c\dot{x}$ dado $Ax \leq b$.

becd4c - dsa/simplex.cpp - 57 lines

```
struct Simplex {
    static constexpr ld EPS = 1e-10;
    static constexpr ld INF = 1e100;
    int m, n;
    vi B, N;
    vector<vector<ld>> D;
    Simplex(vector<vector<ld>> A, vector<ld> b, vector<ld> c)
        : m(sz(b)), n(sz(c)), B(m), N(n + 1), D(m + 2,
        ↪ vector<ld>(n + 2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] =
        ↪ b[i];
        rep(j,0,n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        rep(i,0,m+2) if (i != r)
            rep(j,0,n+2) if (j != s) D[i][j] -= D[r][j] *
            ↪ D[i][s] * inv;
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    } // eb9c6e
    bool simplex(int ph) {
        int x = ph == 1 ? m + 1 : m;
        while (1) {
            int s = -1;
            rep(j,0,n+1) if (!(ph == 2 && N[j] == -1) &&
                (s == -1 || D[x][j] < D[x][s] - EPS ||
                (abs(D[x][j] - D[x][s]) <= EPS && N[j] <
                ↪ N[s]))) s = j;
            if (D[x][s] >= -EPS) return 1;
            int r = -1;
            rep(i,0,m) if (D[i][s] > EPS &&
                (r == -1 || D[i][n + 1] / D[i][s] < D[r][n +
                ↪ 1] / D[r][s] - EPS ||
                (abs(D[i][n + 1] / D[i][s] - D[r][n + 1] /
                ↪ D[r][s]) <= EPS && B[i] < B[r]))) r = i;
            if (r == -1) return 0;
            pivot(r, s);
        }
    } // fee585
    ld solve(vector<ld> &x) {
        int r = 0;
        rep(i,1,m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            pivot(r, n);
            if (!simplex(1) || D[m + 1][n + 1] < -EPS) return
            ↪ -INF;
            rep(i,0,m) if (B[i] == -1) {
                int s = -1;
                rep(j,0,n+1) if (s == -1 || D[i][j] < D[i][s]
                ↪ - EPS ||
                    (abs(D[i][j] - D[i][s]) <= EPS && N[j] <
                    ↪ N[s])) s = j;
                pivot(i, s);
            }
        }
        if (!simplex(2)) return INF;
        x.assign(n, 0);
        rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return D[m][n + 1];
    } // 38c01f
};
```

Wavelet Matrix

Wavelet matrix do array `a`. `rangeKth(wm, H, l, r, k)`. `cntLess(wm, H, l, r, x)`.

399393 - dsa/wavelet-matrix.cpp - 36 lines

```
const int H = 31;
using WM = array<vi, H>;
WM wavMat(vector<ll> a) {
    WM wm;
    int n = sz(a);
    for (int i = H - 1; i >= 0; i--) {
        wm[i].resize(n + 1);
        rep(j, 1, n + 1) wm[i][j] = a[j - 1] >> i & 1;
        stable_partition(all(a), [&](ll x) { return ~x >> i &
        ↪ 1; });
        rep(j, 1, n + 1) wm[i][j] += wm[i][j - 1];
    }
    return wm;
} // 9aeaa7

tuple<int, int, int, int> getSplit(WM &wm, int h, int l, int
↪ r) {
    int a1 = wm[h][l], a0 = l - a1,
        b1 = wm[h][r], b0 = r - b1,
        n = sz(wm[0]) - 1, c = n - wm[h][n];
    return {a0, b0, c + a1, c + b1};
} // f64ad6

ll rangeKth(WM &wm, int h, int l, int r, int k) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    int sl = r0 - l0;
    if (k < sl) return rangeKth(wm, h - 1, l0, r0, k);
    return (1 << (h - 1)) + rangeKth(wm, h - 1, l1, r1, k -
    ↪ sl);
} // 97e884

int cntLess(WM &wm, int h, int l, int r, ll x) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    if (x >> (h - 1) & 1)
        return r0 - l0 + cntLess(wm, h - 1, l1, r1, x);
    return cntLess(wm, h - 1, l0, r0, x);
} // fcb25d
```

Wavelet Matrix++

Wavelet Matrix do array `a`. Retorna `[wm, pos]`, onde `wm` é a wavelet matrix, e `pos[h][i]` diz a posição do elemento `i` do array na altura `h`. `rectOp(wm, l, r, a, b, f)` chama a função `f` em cada intervalo da wavelet matrix que representa os elementos no intervalo `[l, r)` do array que estão em `[a, b)`.

8500de - dsa/wavelet-matrix-extended.cpp - 59 lines

```cpp
const int H = 30;
using WM = array<vi, H>;
pair<WM, WM> wavMat(vi a) {
    WM wm, pos;
    int n = sz(a);
    vi ord(n), b = a;
    iota(all(ord), 0);
    for (int i = H - 1; i >= 0; i--) {
        wm[i].resize(n + 1), pos[i].resize(n);
        rep(j, 1, n + 1) wm[i][j] = a[j - 1] >> i & 1;
        stable_partition(all(ord), [&](int j) { return ~b[j]
        ↪  >> i & 1; });
        stable_partition(all(a), [&](int x) { return ~x >> i &
        ↪  1; });
        rep(j, 1, n + 1) wm[i][j] += wm[i][j - 1];
        rep(j, 0, n) pos[i][ord[j]] = j;
    }
    return {wm, pos};
} // d9e276

tuple<int, int, int, int> getSplit(WM &wm, int h, int l, int
↪  r) {
    int a1 = wm[h][l], a0 = l - a1,
        b1 = wm[h][r], b0 = r - b1,
        n = sz(wm[0]) - 1, c = n - wm[h][n];
    return {a0, b0, c + a1, c + b1};
} // f64ad6

void rectUp(WM &wm, int h, int l, int r, int a, auto f) {
    if (h == 0) {
        f(h, l, r);
        return;
    }
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    if (a >> (h - 1) & 1) rectUp(wm, h - 1, l1, r1, a, f);
    else {
        rectUp(wm, h - 1, l0, r0, a, f);
        f(h - 1, l1, r1);
    }
} // c826ce

void rectDown(WM &wm, int h, int l, int r, int b, auto f) {
    if (h == 0) return;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    if (b >> (h - 1) & 1) {
        f(h - 1, l0, r0);
        rectDown(wm, h - 1, l1, r1, b, f);
    } else rectDown(wm, h - 1, l0, r0, b, f);
} // 7f0baf

void rectOp(WM &wm, int l, int r, int a, int b, auto f) {
    for (int h = H; h; h--) {
        auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
        if ((a ^ b) >> (h - 1) & 1) {
            rectUp(wm, h - 1, l0, r0, a, f);
            rectDown(wm, h - 1, l1, r1, b, f);
            return;
        }
        if (a >> (h - 1) & 1) l = l1, r = r1;
        else l = l0, r = r0;
    }
} // 8c07e7
```

Máximo de sufixo

Mantém pares `(x, y)`, e encontra o maior y para $x \geq X$.

508151 - dsa/suffix-max.cpp - 20 lines

```cpp
template<class K, class V>
struct SuffixMax {
    map<K, V> m;
    V e;
    SuffixMax(V e = V{}): e(e) {}
    void add(K x, V y) {
        auto it = m.lower_bound(x);
        if (it != end(m) && it->se >= y) return;
        it = m.insert_or_assign(it, x, y);
        while (it != begin(m)) {
            auto p = prev(it);
            if (y < p->se) break;
            m.erase(p);
        }
    } // 3a4c71
    V query(K x) {
        auto it = m.lower_bound(x);
        return it == end(m) ? e : it->se;
    } // 15381b
};
```

Grafos

Grafo com sequência de graus

Constrói um grafo a partir de uma sequência de graus. Complexidade: $O(N + M)$.

03e2bc - graph/havel-hakimi.cpp - 38 lines

```cpp
vector<pii> havelHakimi(vi deg) {
    int n = sz(deg), mx = n;
    vector<pii> ed;
    ed.reserve(accumulate(all(deg), 0LL) / 2);
    vi head(n + 1, -1), nxt(n, -1), prv(n, -1), take;
    auto rem = [&](int v) {
        int d = deg[v], p = prv[v], q = nxt[v];
        if (p == -1) head[d] = q;
        else nxt[p] = q;
        if (q != -1) prv[q] = p;
        prv[v] = nxt[v] = -1;
    };
    auto ins = [&](int v) {
        int d = deg[v];
        prv[v] = -1, nxt[v] = head[d];
        if (head[d] != -1) prv[head[d]] = v;
        head[d] = v;
    };
    rep(i,0,n) if (deg[i]) ins(i);
    while (mx && head[mx] == -1) --mx;
    while (mx) {
        int v = head[mx];
        rem(v);
        int d = deg[v];
        take.clear(), take.reserve(d);
        int cur = mx;
        rep(_,0,d) {
            while (cur && head[cur] == -1) --cur;
            int u = head[cur], rem(u);
            take.pb(u);
        }
        for (int u : take) ed.eb(v, u), --deg[u];
        deg[v] = 0;
        for (int u : take) if (deg[u]) ins(u);
        while (mx && head[mx] == -1) --mx;
    }
    return ed;
} // 03e2bc
```

Caminho Euleriano

`adj[v] = {to, id}`. Retorna a ordem de um caminho euleriano em $O(V + E)$.

483eba - graph/eulerian-walk.cpp - 23 lines

```cpp
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s =
↪  0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
            path.pb(v), walk.pb(e);
```

```
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.eb(u, id);
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin()
    ↪  + 1, walk.rend()) };
} // 483eba
```

Árvore de pontes

`comp[v]` = componente de v na árvore de pontes. Ponte uv sse `comp[u] != comp[v]`. $O(V + E)$.

c3d125 - graph/bridges.cpp - 31 lines

```
vi bridges(vector<vi> &g) {
    int n = sz(g), ti = 0, cc = 0;
    vi tin(n), low(n), st, comp;
    comp.assign(n, -1);
    auto make = [&](int v) {
        while (1) {
            int x = st.back(); st.pop_back();
            comp[x] = cc;
            if (x == v) break;
        }
        cc++;
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        tin[v] = low[v] = ++ti;
        st.pb(v);
        int usedPar = 0;
        for (int to : g[v]) {
            if (to == p && !usedPar) { usedPar = 1; continue;
            ↪  }
            if (!tin[to]) {
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] > tin[v]) make(to);
            } else low[v] = min(low[v], tin[to]);
        }
    };
    rep(i,0,n) if (!tin[i]) {
        dfs(dfs, i, -1);
        make(i);
    }
    return comp;
} // c3d125
```

Árvore de bloco e corte

`t[i]` = a quais componentes biconexas o vértice `i` pertence. `t[i].size() > 1` quando `i` é um ponto de articulação. Quando `i >= n`, `t[i]` são os vértices na componente biconexa `i`. $O(V + E)$.

7605c4 - graph/block-cut-tree.cpp - 35 lines

```
vector<vi> bct(vector<vi> &g) {
    int n = sz(g), ti = 0;
    vector<vi> t(n);
    vi num(n), low(n), seen(n);
    vector<pii> st;
    auto add = [&](int a, int b) {
        int id = sz(t), it = id + 1;
        t.pb({});
        while (1) {
            auto [x, y] = st.back(); st.pop_back();
            if (seen[x] != it) seen[x] = it, t[x].pb(id),
            ↪  t[id].pb(x);
            if (seen[y] != it) seen[y] = it, t[y].pb(id),
            ↪  t[id].pb(y);
            if (x == a && y == b) break;
        }
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        num[v] = low[v] = ++ti;
        int ch = 0;
        for (int to : g[v]) if (to != p) {
            if (!num[to]) {
                st.eb(v, to);
                ch++;
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= num[v]) add(v, to);
            } else if (num[to] < num[v]) {
                st.eb(v, to);
                low[v] = min(low[v], num[to]);
            }
        }
        if (p == -1 && !ch) t.pb({v}), t[v].pb(sz(t) - 1);
    };
    rep(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
    return t;
} // 7605c4
```

Numeração st

Encontra uma numeração st no grafo. Assume que o grafo é biconexo após adicionar a aresta st. Complexidade: $O(V + E)$.

207b6a - graph/st-numbering.cpp - 26 lines

```
vi stNumber(vector<vi> &g, int s, int t) {
    int n = sz(g), ti = 0;
    vi in(n, -1), low(n), p(n, -1), by(n), sig(n, -1), num(n);
    auto dfs = [&](auto &&self, int v) -> void {
        in[v] = low[v] = ti, by[ti++] = v;
        auto go = [&](int u) {
            if (u == p[v]) return;
            if (in[u] < 0) p[u] = v, self(self, u), low[v] =
            ↪  min(low[v], low[u]);
            else low[v] = min(low[v], in[u]);
        };
        if (v == s && in[t] < 0) p[t] = s, self(self, t); //
        ↪  virtual edge
        for (int u : g[v]) if (v != s || u != t) go(u);
    };
    dfs(dfs, s);
    list<int> l = {s, t};
    vector<list<int>::iterator> it(n);
    it[s] = l.begin(), it[t] = next(l.begin());
    rep(i,2,n) {
        int v = by[i], q = p[v], x = by[low[v]];
        if (sig[x] < 0) it[v] = l.insert(it[q], v), sig[q] =
        ↪  1;
        else it[v] = l.insert(next(it[q]), v), sig[q] = -1;
    }
    int i = 0;
    for (int v : l) num[v] = i++;
    return num;
} // 207b6a
```

Dominator Tree

`domTree(g, s)` retorna o dominador imediato; `-1` para inalcançáveis. Quase linear.

72f9c4 - graph/dominator-tree.cpp - 34 lines

```
vi domTree(vector<vi> &g, int s) {
    int n = sz(g), t = 0;
    vector<vi> rg(n + 1), bucket(n + 1);
    vi arr(n), rev(n + 1), par(n + 1), sdom(n + 1), dom(n +
    ↪  1), dsu(n + 1), mn(n + 1);
```

```
    auto dfs = [&](auto &&self, int u) -> void {
        arr[u] = ++t; rev[t] = u;
        sdom[t] = dom[t] = dsu[t] = mn[t] = t;
        for (int v : g[u]) {
            if (!arr[v]) self(self, v), par[arr[v]] = arr[u];
            rg[arr[v]].pb(arr[u]);
        }
    };
    auto find = [&](auto &&self, int u, int x = 0) -> int {
        if (u == dsu[u]) return x ? -1 : u;
        int v = self(self, dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[mn[dsu[u]]] < sdom[mn[u]]) mn[u] =
        ↪  mn[dsu[u]];
        return dsu[u] = v, x ? v : mn[u];
    };
    dfs(dfs, s);
    for (int i = t; i; i--) {
        for (int j : rg[i]) sdom[i] = min(sdom[i],
        ↪  sdom[find(find, j)]);
        if (i > 1) bucket[sdom[i]].pb(i);
        for (int j : bucket[i]) {
            int y = find(find, j);
            dom[j] = sdom[y] == sdom[j] ? sdom[j] : y;
        }
        if (i > 1) dsu[i] = par[i];
    }
    rep(i, 2, t + 1) if (dom[i] != sdom[i]) dom[i] =
    ↪  dom[dom[i]];
    vi idom(n, -1);
    rep(i, 1, t + 1) idom[rev[i]] = rev[dom[i]];
    return idom;
} // 72f9c4
```

Árvore geradora mínima direcionada

`dmst(n, r, e)` retorna o custo da arborescência mínima, ou -1; $O(E \log(V))$.

849978 - graph/directed-mst.cpp - 65 lines

```
struct E { int a, b; ll w; };
struct H { int a; ll w, d = 0; H *l = 0, *r = 0; };
void P(H *x) {
    if (!x || !x->d) return;
    x->w += x->d;
    if (x->l) x->l->d += x->d;
    if (x->r) x->r->d += x->d;
    x->d = 0;
} // 089de9
H* M(H *a, H *b) {
    if (!a || !b) return a ? a : b;
    P(a), P(b);
    if (a->w > b->w) swap(a, b);
    a->r = M(a->r, b);
    swap(a->l, a->r);
    return a;
} // 995744
H* pop(H *x) {
    P(x);
    return M(x->l, x->r);
} // 9a6e69

ll dmst(int n, int r, vector<E> es) {
    vector<H*> h(n);
    for (auto e : es) h[e.b] = M(h[e.b], new H{e.a, e.w});
    vi uf(n), se(n, -1);
    iota(all(uf), 0);
    auto f = [&](auto &&f, int x) -> int {
        return uf[x] == x ? x : uf[x] = f(f, uf[x]);
    };
    se[r] = r;
    vector<pair<int, ll>> st;
    ll ans = 0;
    rep(s,0,n) {
        int u = f(f, s);
        if (se[u] >= 0) continue;
        st.clear();
        while (se[u] < 0) {
            se[u] = s;
            while (h[u] && f(f, h[u]->a) == u) h[u] =
            ↪  pop(h[u]);
            if (!h[u]) return LLONG_MAX;
            P(h[u]);
            ll c = h[u]->w;
            ans += c;
            st.eb(u, c);
            u = f(f, h[u]->a);
            if (se[u] == s) {
                H *x = 0;
                int v = u;
                for (;;) {
                    auto [w, c] = st.back();
                    st.pop_back();
                    h[w]->d -= c;
                    x = M(x, h[w]);
                    uf[w] = v;
                    if (w == v) break;
                }
                u = v;
                h[u] = x;
                se[u] = -1;
            }
        }
    }
    return ans;
} // bad540
```

Floresta de grafo funcional

`par[i]` = imagem de `i`. `root[i]` = representante do ciclo/componente de `i`. $O(n)$.

988efe - graph/functional-forest.cpp - 18 lines

```
vi funcForest(vi &f) {
    vi deg(sz(f)), root(sz(f), -1), ord;
    for (int v : f) deg[v]++;
    queue<int> q;
    rep (i, 0, sz(f)) if (!deg[i]) q.push(i);
    while (sz(q)) {
        int v = q.front(); q.pop();
        ord.pb(v);
        if (!--deg[f[v]]) q.push(f[v]);
    }
    rep (i, 0, sz(f)) if (deg[i]) {
        root[i] = i;
        for (int v = f[i]; v != i; v = f[v]) root[v] = i,
        ↪  deg[v] = 0;
        deg[i] = 0;
    }
    for (int i = sz(ord) - 1; i >= 0; --i) root[ord[i]] =
    ↪  root[f[ord[i]]];
    return root;
} // 988efe
```

Componentes fortemente conexas e grafo de condensação

`cnt` = número de SCCs. `comp[v]` = SCC de `v`. `condGraph` = DAG. $O(V + E)$.

2c057a - graph/scc.cpp - 31 lines

```
pair<int, vi> scc(vector<vi> &g) {
    int n = sz(g);
    vector<vi> rg(n);
    rep (v, 0, n) for (int u : g[v]) rg[u].pb(v);
    vi vis(n), ord, comp(n, -1);
    auto dfs1 = [&](auto self, int v) -> void {
        vis[v] = 1;
        for (int u : g[v]) if (!vis[u]) self(self, u);
        ord.pb(v);
    };
    auto dfs2 = [&](auto self, int v, int c) -> void {
```

```
        comp[v] = c;
        for (int u : rg[v]) if (comp[u] < 0) self(self, u, c);
    };
    rep (i, 0, n) if (!vis[i]) dfs1(dfs1, i);
    reverse(all(ord));
    int cnt = 0;
    for (int v : ord) if (comp[v] < 0) dfs2(dfs2, v, cnt++);
    return {cnt, comp};
} // a8c254

vector<vi> condGraph(vector<vi> &g, int cnt, vi &comp) {
    vector<vi> dag(cnt);
    rep (v, 0, sz(g)) for (int u : g[v])
        if (comp[v] != comp[u]) dag[comp[v]].pb(comp[u]);
    rep (i, 0, cnt) {
        sort(all(dag[i]));
        dag[i].erase(unique(all(dag[i])), end(dag[i]));
    }
    return dag;
} // a5cf62
```

2-SAT

`ts.clause(...)` adiciona cláusulas. `ts.solve()` decide. `ts.val[i]` = valor final da variável `i`. $O(V+E)$.

1412b8 - graph/2-sat.cpp - 22 lines

```
struct TwoSat {
    int n;
    vector<vi> g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    int lit(int x, bool v) { return 2 * x + v; }
    void imp(int a, int b) { g[a].pb(b); }
    void clause(int a, bool va, int b, bool vb) {
        int x = lit(a, va), y = lit(b, vb);
        imp(x ^ 1, y);
        imp(y ^ 1, x);
    } // 47d6a6
    bool solve() {
        auto [cnt, comp] = scc(g);
        val.assign(n, 0);
        rep (i, 0, n) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
            val[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return 1;
    } // a54ac0
};
```

Matching bipartido máximo

`hopKarp(g, l, r)` atualiza os matchings `l`/`r` e retorna o tamanho; $O(E\sqrt{V})$.

2d4485 - graph/hopcroft-karp.cpp - 31 lines

```
int hopKarp(vector<vi> &g, vi &l, vi &r) {
    int n = sz(l), m = sz(r);
    vi d(n);
    auto bfs = [&]() {
        queue<int> q;
        rep(i, 0, n) d[i] = l[i] < 0 ? q.push(i), 0 : -1;
        bool f = 0;
        while (sz(q)) {
            int v = q.front(); q.pop();
            for (int u : g[v]) {
                int w = r[u];
                if (w < 0) f = 1;
                else if (d[w] < 0) d[w] = d[v] + 1, q.push(w);
            }
        }
        return f;
    };
    auto dfs = [&](auto &&self, int v) -> bool {
        for (int u : g[v]) {
            int w = r[u];
            if (w < 0 || (d[w] == d[v] + 1 && self(self, w))) {
                l[v] = u, r[u] = v;
                return 1;
            }
        }
        return d[v] = -1, 0;
    };
    int ans = 0;
    while (bfs()) rep(i, 0, n) ans += l[i] < 0 && dfs(dfs, i);
    return ans;
} // 2d4485
```

Max Flow

Dinic

`addEdge(u, v, cap)` e `flow(s, t)` para fluxo máximo.

f40f68 - graph/dinic.cpp - 55 lines

```
struct Dinic {
    struct Edge {
        ll s, t, f, c, r;
    };
    vector<Edge> e;
    vector<vi> adj;
    vi ne, lvl;
    Dinic(int n): adj(n), ne(n), lvl(n) {}
    void addEdge(int a, int b, ll c) {
        adj[a].pb(sz(e));
        e.eb(a, b, 0, c, sz(e) + 1);
        adj[b].pb(sz(e));
        e.eb(b, a, 0, 0, sz(e) - 2);
    } // 598446
    bool bfs(int s, int t) {
        queue<int> q;
        q.push(s);
        fill(all(lvl), -1);
        lvl[s] = 0;
        while (sz(q)) {
            int x = q.front();
            q.pop();
            for (int i: adj[x]) {
                if (e[i].f == e[i].c) continue;
                if (lvl[e[i].t] != -1) continue;
                lvl[e[i].t] = lvl[x] + 1;
                q.push(e[i].t);
            }
        }
        return lvl[t] == -1;
    } // 1625d2
    ll dfs(int x, int t, ll f) {
        if (f == 0) return 0;
        if (x == t) return f;
        for (int &te = ne[x]; te < sz(adj[x]); te++) {
            int i = adj[x][te], y = e[i].t;
            if (lvl[y] != lvl[x] + 1) continue;
            if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
                e[i].f += df;
                e[i ^ 1].f -= df;
                return df;
            }
        }
        return 0;
    } // d7b05b
    ll flow(int s, int t, ll mx = LLONG_MAX) {
        ll mf = 0;
        while (true) {
            if (bfs(s, t)) break;
            fill(all(ne), 0);
            while (ll f = dfs(s, t, mx - mf)) mf += f;
        }
        return mf;
    } // 53644f
};
```

Min Cost Flow

MCMF

`addEdge(u, v, cap, cost)` e `flow(s, t)` para min-cost max-flow.

e07252 - graph/mcf.cpp - 58 lines

```cpp
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n;
    vector<vector<E>> g;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].eb(v, sz(g[v]), cap, cost);
        g[v].eb(u, sz(g[u]) - 1, 0, -cost);
    } // 82791b
    pll flow(int s, int t, ll k = 4e18) {
        const ll INF = 4e18;
        ll fl = 0, cost = 0;
        vector<ll> p(n, INF), d(n), f(n);
        vi pv(n), pe(n), inq(n);
        queue<int> q;
        p[s] = 0, q.push(s), inq[s] = 1;
        while (sz(q)) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            rep(i,0,sz(g[u])) {
                auto &e = g[u][i];
                if (e.cap && p[e.to] > p[u] + e.cost) {
                    p[e.to] = p[u] + e.cost;
                    if (!inq[e.to]) q.push(e.to), inq[e.to] =
                    ↪ 1;
                }
            }
        }
        rep(i,0,n) if (p[i] == INF) p[i] = 0;
        while (fl < k) {
            fill(all(d), INF);
            priority_queue<pll, vector<pll>, greater<pll>> pq;
            d[s] = 0, f[s] = k - fl, pq.emplace(0, s);
            while (sz(pq)) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                rep(i,0,sz(g[u])) {
                    auto &e = g[u][i];
                    if (!e.cap) continue;
                    ll nd = du + e.cost + p[u] - p[e.to];
                    if (d[e.to] > nd) {
                        d[e.to] = nd, pv[e.to] = u, pe[e.to] =
                        ↪ i;
                        f[e.to] = min(f[u], e.cap);
                        pq.emplace(nd, e.to);
                    }
                }
            }
            if (d[t] == INF) break;
            rep(i,0,n) if (d[i] < INF) p[i] += d[i];
            ll x = f[t];
            fl += x, cost += x * p[t];
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= x, g[v][e.rev].cap += x;
            }
        }
        return {fl, cost};
    } // e0d612
};
```

Matching bipartido com menor custo

`cost` = custo mínimo. `match[i]` = coluna escolhida para a linha `i`. $(n <= m)$ $O(n^2m)$.

c8c7c0 - graph/hungarian.cpp - 33 lines

```cpp
pair<ll, vi> hungarian(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]);
    const ll INF = 4e18;
    vector<ll> u(n + 1), v(m + 1), minv(m + 1);
    vi p(m + 1), way(m + 1), ans(n);
    rep(i, 1, n + 1) {
        p[0] = i;
        int j0 = 0;
        fill(all(minv), INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    rep(j, 1, m + 1) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans};
} // c8c7c0
```

Matching Geral

Encontra o tamanho do matching máximo em um grafo geral. Complexidade: $O(N^3)$.

b2c903 - graph/matching-size.cpp - 16 lines

```cpp
int matchingSize(int n, vector<pii> &es, int its = 2) {
    uniform_int_distribution<int> dist(1, mod - 1);
    int ans = 0;
    rep(it,0,its) {
        vector<vector<ll>> a(n, vector<ll>(n));
        for (auto [u, v] : es) if (u != v) {
            int x = dist(rng);
            a[u][v] += x;
            if (a[u][v] >= mod) a[u][v] -= mod;
            a[v][u] -= x;
            if (a[v][u] < 0) a[v][u] += mod;
        }
        ans = max(ans, gauss(a) / 2);
    }
    return ans;
} // b2c903
```

Encontrar Matching Geral

Encontra o matching máximo em um grafo geral. Retorna o matching. Complexidade: $O(N^3)$.

35862b - graph/matching.cpp - 51 lines

```cpp
vi matching(vector<vi> &g) {
    int n = sz(g), T = 0;
    vi m(n, -1), p(n), b(n), q(n), u(n), w(n), l(n);
    auto F = [&](int x, int y) {
        for (++T;; x = p[m[x]]) {
            x = b[x];
            l[x] = T;
            if (m[x] < 0) break;
        }
        for (;; y = p[m[y]]) {
            y = b[y];
            if (l[y] == T) return y;
        }
    };
    auto G = [&](int x, int c, int y) {
```

```
        for (; b[x] != c; x = p[m[x]])
            w[b[x]] = w[b[m[x]]] = 1, p[x] = y, y = m[x];
    };
    rep(s, 0, n) if (m[s] < 0) {
        fill(all(u), 0), fill(all(p), -1), iota(all(b), 0);
        int a = 0, z = 0, x = -1;
        q[z++] = s, u[s] = 1;
        while (a < z && x < 0) {
            int v = q[a++];
            for (int y : g[v]) {
                if (b[v] == b[y] || m[v] == y)
                    continue;
                if (y == s || (~m[y] && ~p[m[y]])) {
                    int c = F(v, y);
                    fill(all(w), 0);
                    G(v, c, y);
                    G(y, c, v);
                    rep(i, 0, n) if (w[b[i]])
                        b[i] = c, !u[i] && (u[i] = 1, q[z++] =
                        ↪ i);
                } else if (!~p[y]) {
                    p[y] = v;
                    if (!~m[y]) {
                        x = y;
                        break;
                    }
                    y = m[y], u[y] = 1, q[z++] = y;
                }
            }
        }
        while (~x) {
            int y = p[x], v = m[y];
            m[x] = y, m[y] = x, x = v;
        }
    }
    return m;
} // 35862b
```

Árvores

LCA

`LCA lca(g, root)` e `lca.lca(a, b);` build $O(n \log n)$, query $O(1)$.

1d44d9 - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a,
↪ b); }>;

struct LCA {
    int t = 0;
    vi tm, path, ret;
    RMQ rmq;
    LCA(vector<vi> &g, int r = 0) : tm(g.size()) {
        dfs(g, r, -1);
        rmq.build(ret);
    }
    void dfs(vector<vi> &g, int x, int pre) {
        tm[x] = t++;
        for (int y: g[x]) if (y != pre) {
            path.pb(x), ret.pb(tm[x]);
            dfs(g, y, x);
        }
    } // 2e7ca5
    int lca(int x, int y) {
        if (x == y) return x;
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

HLD

Heavy-light decomposition. Complexidade: Construção em $O(n)$, caminhos em $O(\log(n))$.

4478db - tree/hld.cpp - 39 lines

```
struct HLD {
    vi cnt, nxt, par, tin, tout, d;
    int T = 0;
    HLD(vector<vi> g, int r = 0) :
        cnt(sz(g)), nxt(sz(g)), par(sz(g), -1),
        tin(sz(g)), tout(sz(g)), d(sz(g)) {
        auto dfs1 = [&](auto &&self, int u) -> void {
            cnt[u] = 1, nxt[u] = u;
            if (par[u] != -1) g[u].erase(find(all(g[u]),
            ↪ par[u]));
            for (int &v: g[u]) {
                par[v] = u, d[v] = d[u] + 1;
                self(self, v);
                cnt[u] += cnt[v];
                if (cnt[v] > cnt[g[u][0]]) swap(v, g[u][0]);
            }
        } // 0c9ec6;
        auto dfs2 = [&](auto &&self, int u) -> void {
            tin[u] = T++;
            for (int v: g[u]) {
                if (v == g[u][0]) nxt[v] = nxt[u];
                self(self, v);
            }
            tout[u] = T;
        } // a64312;
        dfs1(dfs1, r), dfs2(dfs2, r);
    }
    vector<pii> path(int u, int v) {
        vector<pii> a, b;
        while (nxt[u] != nxt[v]) {
            if (d[nxt[u]] < d[nxt[v]])
                b.eb(tin[nxt[v]], tin[v]), v = par[nxt[v]];
            else
                a.eb(tin[u], tin[nxt[u]]), u = par[nxt[u]];
        }
        (d[u] < d[v] ? b : a).eb(tin[u], tin[v]);
        a.insert(end(a), rbegin(b), rend(b));
        return a;
    } // 32f9ea
};
```

Matching em Árvores

`treeMatching(g, root)` retorna o parceiro de cada vértice em $O(n)$.

6e0886 - tree/tree-matching.cpp - 17 lines

```
vi treeMatching(vector<vi> &g, int root = 0) {
    int n = sz(g);
    vi par(n, -1), ord{root}, mat(n, -1);
    rep(i,0,sz(ord)) {
        int v = ord[i];
        for (int u : g[v]) if (u != par[v]) {
            par[u] = v;
            ord.pb(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = ord[i], p = par[v];
        if (mat[v] == -1 && mat[p] == -1)
            mat[v] = p, mat[p] = v;
    }
    return mat;
} // 6e0886
```

Árvore virtual

Encontra as arestas da árvore virtual do conjunto no formato `[par, child]`. $O(k \log(n))$.

57ab2b - tree/virtual-tree.cpp - 17 lines

```
vector<pii> virtree(vector<vi> &t, LCA &lca, vi &tin, vi
↪ &tout, vi &v) {
```

```
    if (v.empty()) return {};
    auto cmp = [&](int a, int b) { return tin[a] < tin[b]; };
    sort(all(v), cmp);
    int m = sz(v);
    rep(i,0,m-1) v.pb(lca.lca(v[i], v[i+1]));
    sort(all(v), cmp);
    v.erase(unique(all(v)), v.end());
    vector<pii> e;
    vi st = {v[0]};
    rep(i,1,sz(v)) {
        while (tout[st.back()] <= tin[v[i]]) st.pop_back();
        e.eb(st.back(), v[i]);
        st.pb(v[i]);
    }
    return e;
} // 57ab2b
```

Strings

KMP

`kmp(s)` retorna a prefix function em $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Z-function

`zFunc(s)` retorna o tamanho do match com o prefixo em cada posição, em $O(n)$.

fc69f3 - strings/z-function.cpp - 12 lines

```
vi zFunc(string s) {
    int n = sz(s), l = 0, r = 0;
    vi z(n);
    if (!n) return z;
    rep(i, 1, n) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    z[0] = n;
    return z;
} // fc69f3
```

Array de sufixos

`sufArr(s)` retorna os índices dos sufixos em ordem lexicográfica em $O(n \log n)$.

f69f29 - strings/suffix-array.cpp - 27 lines

```
vi sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s);
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        };
        srt(d); srt(c);
        e[sa[0]] = 1;
        rep(i,1,n) {
            int a = sa[i-1], b = sa[i];
            e[b] = e[a] + (c[a] != c[b] || d[a] != d[b]);
        }
        swap(c, e);
    }
    return sa;
} // f69f29
```

Maior prefixo comum (LCP)

`lcpArr(s, sa)` calcula o LCP entre sufixos adjacentes em $O(n)$.

558ffb - strings/lcp.cpp - 12 lines

```
vi lcpArr(string &s, vi &sa) {
    int n = sz(s), k = 0;
    vi rnk(n), lcp(n);
    rep(i,0,n) rnk[sa[i]] = i;
    rep(i,0,n) if (rnk[i]) {
        int j = sa[rnk[i] - 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k])
        ↪  k++;
        lcp[rnk[i]] = k;
        if (k) k--;
    }
    return lcp;
} // 558ffb
```

Árvore de sufixos

`ST(s, sa, lcp)` constrói a árvore de sufixos a partir de `sa` e `lcp` em $O(n)$.

3e9f4e - strings/suffix-tree.cpp - 31 lines

```
struct ST {
    struct N {
        int l, r, p, suf;
        vi ch;
    };
    vector<N> t;
    vi dep;
    int nn(int l, int r, int p, int suf = -1) {
        t.pb({l, r, p, suf, {}});
        dep.pb(p == -1 ? 0 : dep[p] + r - l);
        if (p != -1) t[p].ch.pb(sz(t) - 1);
        return sz(t) - 1;
    } // aa5bfe
    ST(string& s, vi& sa, vi& lcp) {
        int n = sz(s), rt = nn(0, 0, -1), last = rt;
        rep(i,0,n) {
            int x = sa[i], k = i ? lcp[i] : 0;
            while (dep[last] > k) last = t[last].p;
            if (dep[last] < k) {
                int y = t[last].ch.back(), z = nn(t[y].l,
                ↪  t[y].l + k - dep[last], last);
                t[last].ch.back() = z;
                t[y].p = z;
                t[y].l += k - dep[last];
                t[z].ch.pb(y);
                dep[z] = k;
                last = z;
            }
            last = nn(x + k, n, last, x);
        }
    }
};
```

Automato de sufixos

`SAM sam; extend(c)` monta o automato em $O(n)$ total.

a67d77 - strings/suffix-automaton.cpp - 40 lines

```cpp
struct SAM {
    struct N {
        int len = 0, link = -1;
        array<int, 26> to;
        ll cnt = 0; // endpos count after calcCnt()
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    int last = 0;
    void extend(char c) {
        int x = c - 'a', cur = sz(t), p = last;
        t.eb();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1;
        for (; p != -1 && t[p].to[x] == -1; p = t[p].link)
        ↪  t[p].to[x] = cur;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].to[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int cl = sz(t);
                t.pb(t[q]);
                t[cl].len = t[p].len + 1;
                t[cl].cnt = 0;
                for (; p != -1 && t[p].to[x] == q; p =
                ↪  t[p].link) t[p].to[x] = cl;
                t[q].link = t[cur].link = cl;
            }
        }
        last = cur;
    } // 1f27c6
    vi ord() {
        int mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vi cnt(mx + 1), ord(sz(t));
        for (auto &u : t) cnt[u.len]++;
        rep(i,1,mx+1) cnt[i] += cnt[i - 1];
        for (int i = sz(t) - 1; i >= 0; i--)
        ↪  ord[--cnt[t[i].len]] = i;
        return ord;
    } // a8d426
};
```

Encontrando os palíndromos máximos em uma string

`manacher(s)` retorna `{d1, d2}` com raios ímpar/par em $O(n)$.

59be6a - strings/manacher.cpp - 17 lines

```cpp
pair<vi, vi> manacher(string& s) {
    int n = sz(s);
    vi d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i +
        ↪  k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] ==
        ↪  s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2}; // {odd, even}
} // 59be6a
```

Aho-Corasick

`AC: add, build e next` para alfabeto 'a'-'z'. Construção/consulta lineares no total.

05f5f7 - strings/aho-corasick.cpp - 38 lines

```cpp
struct AC {
    static constexpr int A = 26;
    struct N {
        array<int, A> to{};
        int link = 0;
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    vi ord;
    int add(string& s) {
        int v = 0;
        for (char c : s) {
            int x = c - 'a';
            if (t[v].to[x] == -1) t[v].to[x] = sz(t), t.eb();
            v = t[v].to[x];
        }
        return v;
    } // bd83ba
    void build() {
        queue<int> q;
        rep(c,0,A) {
            int u = t[0].to[c];
            if (u == -1) t[0].to[c] = 0;
            else q.push(u);
        }
        ord.clear();
        while (sz(q)) {
            int v = q.front(); q.pop();
            ord.pb(v);
            rep(c,0,A) {
                int& u = t[v].to[c];
                if (u == -1) u = t[t[v].link].to[c];
                else t[u].link = t[t[v].link].to[c],
                ↪  q.push(u);
            }
        }
    } // 5e19d4
    int next(int v, char c) { return t[v].to[c - 'a']; }
};
```

Decomposição de Lyndon

Encontra a decomposição de Lyndon de uma string em $O(n)$. Retorna os índices de split $(0, ..., n)$.

751105 - strings/lyndon.cpp - 14 lines

```cpp
vi lyndon(string s) {
    int n = sz(s);
    vi p = {0};
    for (int i = 0; i < n;) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) p.pb(i += j - k);
    }
    return p;
} // 751105
```

Encontrar runs em uma string

Encontra todas as runs em uma string em $O(n \log n)$.

934209 - strings/runs.cpp - 38 lines

```cpp
using tiii = tuple<int, int, int>;

vector<tiii> getRuns(string &S) {
    int N = sz(S);
    vector<vector<pii>> B(N + 1);
    auto go = [&](string &l, string &r, int M, int o) {
        int n = sz(l), m = sz(r);
        string s = l, t = r + l + r;
        reverse(all(s));
        auto z = zFunc(s), w = zFunc(t);
        rep(p,1,n+1) {
            int a = p == n ? n : min(p + z[p], n), b = min(w[n
            ↪  + m - p], m);
            if (a + b >= 2 * p) B[p].eb(M - (o ? b : a), M +
            ↪  (o ? a : b));
        }
    };
    auto dfs = [&](auto &&self, int L, int R) -> void {
        if (R - L < 2) return;
        int M = (L + R) >> 1;
        self(self, L, M), self(self, M, R);
        string l(begin(S) + L, begin(S) + M), r(begin(S) + M,
        ↪  begin(S) + R);
        go(l, r, M, 0);
        reverse(all(l)), reverse(all(r));
        go(r, l, M, 1);
    };
    dfs(dfs, 0, N);
    vector<tiii> res;
    set<pii> done;
    rep(p,0,N+1) {
        auto &v = B[p];
        sort(all(v), [](pii a, pii b) { return a.fi == b.fi ?
        ↪  a.se > b.se : a.fi < b.fi; });
        int R = -1;
        for (auto [l, r] : v) if (R < r) {
            R = r;
            if (done.emplace(l, r).se) res.eb(p, l, r);
        }
    }
    return res;
} // d4dc26
```

Geometria

Base

673f55 - geometria/core.cpp - 9 lines

```cpp
using Pt = complex<ll>;
#define xx real()
#define yy imag()

ll dot(Pt a, Pt b) { return (conj(a) * b).xx; }
ll cross(Pt a, Pt b) { return (conj(a) * b).yy; }
Pt perp(Pt a) { return Pt(-a.yy, a.xx); }
const ld EPS = 1e-9;
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
```

Interseção de retas

`lineInter(a, b, c, d)` para retas dadas por dois pontos; $O(1)$.

2fc473 - geometria/line-intersection.cpp - 3 lines

```cpp
Pt lineInter(Pt a, Pt b, Pt c, Pt d) {
    return a + (b - a) * cross(c - a, d - c) / cross(b - a, d
    ↪  - c);
} // 2fc473
```

Interseção de segmentos

`segInter` e `properInter` para dois segmentos, em $O(1)$.

f6de72 - geometria/segment-intersection.cpp - 16 lines

```cpp
bool segInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    if (ab_c * ab_d < 0 && cd_a * cd_b < 0) return 1;
    if (!ab_c && sgn(dot(c - a, c - b)) <= 0) return 1;
    if (!ab_d && sgn(dot(d - a, d - b)) <= 0) return 1;
    if (!cd_a && sgn(dot(a - c, a - d)) <= 0) return 1;
    if (!cd_b && sgn(dot(b - c, b - d)) <= 0) return 1;
    return 0;
} // 4f106a

bool properInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
} // 66a13a
```

Área de um polígono

`polyArea(p)` retorna a área orientada multiplicada por 2, em $O(n)$.

b23e06 - geometria/polygon-area.cpp - 5 lines

```cpp
ll polyArea(vector<Pt> &pt) { // ld: change ret type to ld
    ll n = sz(pt), s = 0; // ld: change to ld s
    rep (i, 0, n) s += cross(pt[i], pt[(i + 1) % n]);
    return s;
} // b23e06
```

Verificar se um ponto está dentro de um polígono convexo

Point-in-convex-polygon em $O(\log n)$.

1ce015 - geometria/point-in-polygon.cpp - 10 lines

```cpp
bool pointInPoly(vector<Pt> &pt, Pt q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1; // ld: ld
    ↪  t = border ? EPS : 0
    auto v = [&](int i) { return cross(q - pt[0], pt[i] -
    ↪  pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m; // ld: use v(m) > EPS
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t; // ld: use the
    ↪  ld t above
} // 1ce015
```

Convex hull

`convHull(pt)` retorna os índices dos vértices do hull em $O(n \log n)$.

b43fd9 - geometria/convex-hull.cpp - 22 lines

```cpp
vi convHull(vector<Pt> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                Pt a = pt[st[m - 1]], b = pt[st[m - 2]], c =
                ↪  pt[i];
                if (cross(b - a, c - a) < 0) break; // 1) >
                ↪  for clockwise, <= to include non-vertices
```

```
                st.pop_back();
            }
            st.pb(i);
        }
        st.pop_back();
        h.insert(h.end(), all(st));
    };
    iota(all(ord), 0);
    auto top = [](auto a) { return pair(a.xx, a.yy); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) >
    ↪  top(pt[j]); });
    add(), reverse(all(ord)), add();
    return h;
} // b43fd9 // 1) ld: use sgn(cross(...)) < 0
```

Menor distância euclideana

Closest pair: retorna um par de índices em $O(n \log n)$.

0c44eb - geometry/closest-pair.cpp - 20 lines

```
bool cmpX(Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx :
↪  a.yy < b.yy; }

pair<Pt,Pt> closestPair(vector<Pt> p) {
    sort(all(p), cmpX);
    set<pair<ll,ll>> s;
    pair<Pt,Pt> ret{p[0], p[1]};
    ll best = norm(p[0] - p[1]);
    for (int l = 0, i = 0; i < sz(p); i++) {
        ll d = sqrtl((ld)best) + 1;
        while (p[i].xx - p[l].xx > d) s.erase({p[l].yy,
        ↪  p[l].xx}), l++;
        auto it1 = s.lower_bound({p[i].yy - d, LLONG_MIN});
        auto it2 = s.upper_bound({p[i].yy + d, LLONG_MAX});
        for (auto it = it1; it != it2; ++it) {
            Pt q(it->se, it->fi);
            if (ll cur = norm(p[i] - q); cur < best) best =
            ↪  cur, ret = {p[i], q};
        }
        s.insert({p[i].yy, p[i].xx});
    }
    return ret;
} // 5f8da5
```

Interseção de círculo e reta

`circLineInter(c, r, a, d)` retorna 0, 1 ou 2 pontos, em $O(1)$.

21e90b - geometry/circle-line.cpp - 8 lines

```
vector<Pt> circLineInter(Pt c, ld r, Pt a, Pt d) {
    Pt f = a + d * dot(c - a, d) / norm(d);
    ld h2 = r * r - norm(c - f);
    if (sgn(h2) < 0) return {};
    ld dt = sqrt(max((ld)0, h2)) / abs(d);
    if (!sgn(h2)) return {f};
    return {f - dt * d, f + dt * d};
} // 21e90b
```

Interseção de círculos

`circCircInter(c1, r1, c2, r2)` retorna 0, 1 ou 2 pontos, em $O(1)$.

d3b1f8 - geometry/circle-circle.cpp - 12 lines

```
vector<Pt> circCircInter(Pt c1, ld r1, Pt c2, ld r2) {
    Pt d = c2 - c1;
    ld dist = abs(d);
    if (!sgn(dist)) return {};
    ld a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    ld h2 = r1 * r1 - a * a;
    if (sgn(h2) < 0) return {};
    Pt mid = c1 + d * (a / dist);
    if (!sgn(h2)) return {mid};
    Pt p = perp(d) * (sqrt(max((ld)0, h2)) / dist);
    return {mid - p, mid + p};
} // d3b1f8
```

Tangentes comuns a dois círculos

`circTangents(c1, r1, c2, r2)` retorna pares de tangência, em $O(1)$.

0d00d5 - geometry/circle-tangents.cpp - 14 lines

```
vector<pair<Pt, Pt>> circTangents(Pt c1, ld r1, Pt c2, ld r2)
↪  {
    vector<pair<Pt, Pt>> res;
    for (int s: {1, -1}) {
        ld r = s * r2, dr = r1 - r;
        Pt d = c2 - c1;
        ld l2 = norm(d), h2 = max((ld)0, l2 - dr * dr);
        if (sgn(l2 - dr * dr) < 0) continue;
        for (int t: {1, -1}) {
            Pt n = (dr * d + perp(d) * (t * sqrt(h2))) / l2;
            res.pb({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // 0d00d5
```

Menor cobertura circular

Encontra o menor círculo que contém todos os pontos.

fd30f4 - geometry/enclosing-circle.cpp - 33 lines

```
struct C {
    Pt o;
    ld r;
};

bool in(C c, Pt p) { return abs(p - c.o) <= c.r + EPS; }

C circ(Pt a, Pt b) { return {(a + b) / (ld)2, abs(a - b) / 2};
↪  }

C circ(Pt a, Pt b, Pt c) {
    if (fabsl(cross(b - a, c - a)) < EPS) {
        C x = circ(a, b), y = circ(a, c), z = circ(b, c);
        if (in(x, c)) return x;
        if (in(y, b)) return y;
        return z;
    }
    Pt o = a + perp((b - a) * norm(c - a) - (c - a) * norm(b -
    ↪  a))
        / cross(b - a, c - a) / (ld)2;
    return {o, abs(a - o)};
} // a6aab7

C mec(vector<Pt> p) {
    shuffle(all(p), rng);
    C c{{0, 0}, -1};
    rep(i,0,sz(p)) if (!in(c, p[i])) {
        c = {p[i], 0};
        rep(j,0,i) if (!in(c, p[j])) {
            c = circ(p[i], p[j]);
            rep(k,0,j) if (!in(c, p[k])) c = circ(p[i], p[j],
            ↪  p[k]);
        }
    }
    return c;
} // effcbe
```

Soma Minkowski de polígonos convexos

`minkowski(a, b)` para convexos em sentido anti-horário, em $O(|a| + |b|)$.

09f042 - geometry/minkowski.cpp - 17 lines

```
vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) {
    auto norm = [&](vector<Pt>& p) {
        auto lt = [](Pt a, Pt b) { return a.xx != b.xx ? a.xx
        ↪  < b.xx : a.yy < b.yy; };
        rotate(p.begin(), min_element(all(p), lt), p.end());
        p.pb(p[0]), p.pb(p[1]);
    };
    int n = sz(a), m = sz(b);
    norm(a), norm(b);
    vector<Pt> c;
    for (int i = 0, j = 0; i < n || j < m; ) {
        c.pb(a[i] + b[j]);
        ll z = cross(a[i + 1] - a[i], b[j + 1] - b[j]); // ld:
        ↪  keep z in ld
        if (z >= 0 && i < n) i++; // ld: use z >= -EPS
        if (z <= 0 && j < m) j++; // ld: use z <= EPS
    }
    return c;
} // 09f042 //ld: a.xx != b.xx -> sgn(a.xx - b.xx)
```

Corte de Polígono

Corte de polígono por semiplano em $O(n)$.

a60a48 - geometry/polygon-cut.cpp - 10 lines

```
vector<Pt> polyCut(vector<Pt> p, Pt a, Pt b) {
    vector<Pt> q;
    rep(i,0,sz(p)) {
        Pt c = p[i], d = p[(i + 1) % sz(p)];
        int sc = sgn(cross(b - a, c - a)), sd = sgn(cross(b -
        ↪  a, d - a));
        if (sc >= 0) q.pb(c);
        if (sc * sd < 0) q.pb(lineInter(a, b, c, d));
    }
    return q;
} // a60a48
```

Fórmulas, Teoremas e Fatos

Contagem

Estrelas e Barras. O número de soluções inteiras não negativas de

$$x_1 + \cdots + x_k = n$$

é

$$\binom{n+k-1}{k-1}.$$

O número de soluções inteiras positivas é

$$\binom{n-1}{k-1}.$$

Número de Catalan.

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

Número de Narayana. A quantidade de caminhos de $(0,0)$ a (N,N) que não passam acima da diagonal e com exatamente M picos locais é

$$\frac{1}{N}\binom{N}{M}\binom{N}{M-1}$$

Lema dos ciclos. Dada uma sequência $A_1, A_2, \ldots, A_n$ de números tais que $A_i \leq 1$ e $K = \sum A_i \geq 0$, o número de rotações cíclicas de A tais que cada soma de prefixo é positiva é exatamente K.

Desarranjos.

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$$

e

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

Funções de parking. O número de sequências $a = (a_1, a_2, \ldots, a_m)$ tais que $\forall i, 1 \leq a_i \leq n$, e para $k = 1, 2, \ldots, n$, o número de índices i que satisfazem $a_i \leq k$ é pelo menos $m - n + k$ é $(n+1-m)(n+1)^{m-1}$

Números de Stirling do primeiro tipo. $\begin{bmatrix} n \\ k \end{bmatrix}$ é o número de permutações de tamanho n com exatamente k ciclos.

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix}$$

Números de Stirling do segundo tipo. $\begin{Bmatrix} n \\ k \end{Bmatrix}$ é o número de formas de dividir os inteiros de 1 a n em k caixas não vazias e indistinguíveis.

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k\begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

Forma fechada:

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

Fatoriais subindo Para $x^{\overline{n}} = x(x+1)(x+2)\ldots(x+n-1)$, temos que

$$x^{\overline{n}} = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

e

$$x^n = \sum_{k=0}^{n}(-1)^{n-k}\begin{Bmatrix} n \\ k \end{Bmatrix} x^{\overline{k}}$$

Números de Bell.

$$B_n = \sum_{k=0}^{n} S(n,k)$$

Somas e sequências

Somas de potências comuns.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Convolução binomial (Vandermonde).

$$\sum_k \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}.$$

Somas de Newton. Dado um polinômio $a_n x^n + \cdots + a_1 x + a_0$ com raízes $x_1, x_2, \ldots, x_n$, seja $p_i = x_1^i + x_2^i + \cdots + x_n^i$.

Temos as seguintes relações:

$$a_n p_1 + a_{n-1} = 0$$
$$a_n p_2 + a_{n-1} p_1 + 2 * a_{n-2} = 0$$
$$\vdots$$
$$a_n p_k + a_{n-1} p_{k-1} + \cdots + k a_{n-k} = 0$$

com $a_i = 0$ para $i < 0$.

Combinatória e Otimização

Teorema de Dilworth. Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.

Erdős–Szekeres. Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.

Teorema de Mirsky. Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.

Máximo número de intervalos disjuntos. Ordenar os intervalos por extremo direito crescente e escolher gulosamente produz um conjunto máximo de intervalos dois a dois disjuntos.

Mínimo de pontos para perfurar intervalos. Ordenar os intervalos por extremo direito crescente e escolher repetidamente o extremo direito do primeiro intervalo ainda não coberto produz um conjunto mínimo de pontos que intersecta todos os intervalos.

Intervalos: mínimo de pontos = máximo de disjuntos. Para uma família de intervalos na reta, o número mínimo de pontos necessários para intersectar todos os intervalos é igual ao número máximo de intervalos dois a dois disjuntos.

Intervalos dois a dois intersectantes têm interseção comum. Para intervalos na reta, se todo par intersecta, então todos compartilham um ponto em comum. Isso falha em dimensões maiores, mas é muito útil em 1D.

Maior soma com descarte. Se temos um array $A = (a_0, a_1, \ldots, a_{n-1})$, e podemos escolher alguns a_i, mas para cada a_i escolhido, precisamos descartar algum a_i que veio antes, a maior soma é obtida mantendo um multiset inserindo todos os valores até agora, e descartando o menor valor do multiset em cada índice par.

Teoria dos Números

Truques de floor/ceiling.

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor \qquad (a, b > 0)$$

$$\left\lceil \frac{a}{b} \right\rceil = \frac{a+b-1}{b}$$

em aritmética inteira, para $a, b > 0$.

Contagem / soma de divisores. Se

$$n = \prod p_i^{e_i},$$

então

$$d(n) = \prod (e_i + 1),$$

$$\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

Triplas pitagóricas. Todas as soluções inteiras primitivas de

$$a^2 + b^2 = c^2$$

são exatamente

$$a = m^2 - n^2, \qquad b = 2mn, \qquad c = m^2 + n^2,$$

para inteiros $m > n \geq 1$ com

$$\gcd(m, n) = 1 \quad \text{e} \quad m \not\equiv n \pmod 2.$$

Todas as soluções inteiras são obtidas multiplicando uma solução primitiva por $k \geq 1$.

Somas de três quadrados. Um inteiro não negativo n pode ser escrito como

$$n = x^2 + y^2 + z^2$$

com x, y, z inteiros sse

$$n \neq 4^a(8b + 7)$$

para todos os inteiros $a, b \geq 0$.

Teorema de Lucas. Para primo p, se

$$n = \sum n_i p^i, \qquad k = \sum k_i p^i,$$

então

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p.$$

Teorema de Kummer. O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n-k$ na base p.

LTE (versão comum). Se p é um primo ímpar, $p \mid (a-b)$ e $p \nmid ab$, então

$$v_p(a^n - b^n) = v_p(a-b) + v_p(n).$$

Além disso, para n ímpar e $p \mid (a+b)$,

$$v_p(a^n + b^n) = v_p(a+b) + v_p(n).$$

Inversão de Möbius. Se

$$g(n) = \sum_{d|n} f(d),$$

então

$$f(n) = \sum_{d|n} \mu(d)\, g\Big(\frac{n}{d}\Big).$$

Existência de raiz primitiva. Existem raízes primitivas módulo n sse

$$n \in \{1, 2, 4, p^k, 2p^k\},$$

onde p é um primo ímpar.

Grafos e Jogos

Teorema do Casamento de Hall. Um grafo bipartido (L, R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \geq |S|.$$

Teorema de König. Em um grafo bipartido,

matching máximo = cobertura mínima.

Condições para trilha/circuito euleriano (direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente fracamente conexa. Então:

- existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice;
- existe trilha euleriana sse exatamente um vértice tem $\deg^+(v) = \deg^-(v)+1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.

Condições para trilha/circuito euleriano (não direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente conexa. Então:

- existe circuito euleriano sse todo vértice tem grau par;
- existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.

Critério de 2-SAT. Uma instância de 2-SAT é satisfatível sse, para toda variável x, os literais x e $\neg x$ estão em SCCs diferentes do grafo de implicação.

Restrições de diferença $\leftrightarrow$ menores caminhos. Um sistema de desigualdades da forma

$$x_v - x_u \leq w$$

pode ser modelado por uma aresta $u \to v$ de peso w. Então:

- o sistema é viável sse não existe ciclo negativo;
- uma solução viável é dada pelas distâncias a partir de uma super-fonte.

Cobertura por caminhos em DAG via matching. O número mínimo de caminhos disjuntos em vértices que cobrem todos os vértices de um DAG é

$$n - \text{(matching máximo na expansão bipartida do DAG)}.$$

Teorema Matrix-Tree de Kirchhoff. O número de árvores geradoras de um grafo é igual a qualquer cofator de sua matriz Laplaciana. (A matriz de graus - a matriz de adjacência).

Jogo UGV. Considere o jogo de mover ao longo de arestas de um grafo sem poder repetir vértices, e o jogador que não puder mover perde. O segundo jogador tem uma estratégia vencedora sse existe um matching máximo no grafo que deixa o vértice inicial insaturado.

Strings

Teorema de Fine–Wilf. Se uma string tem períodos p e q, e seu tamanho é pelo menos

$$p + q - \gcd(p, q),$$

então ela também tem período $\gcd(p, q)$.

Substrings distintas. Usando array de sufixos:

$$\frac{n(n+1)}{2} - \sum lcp[i]$$

Usando autômato de sufixos:

$$\sum len(i) - len(link(i))$$

Notas de automato de sufixos. O automato de sufixos é um DAG. Cada caminho a partir da raiz representa uma substring. Cada nó representa um conjunto de `endpos`. Para obter o tamanho dos conjuntos de endpos, basta propagar os `cnt` por suffix links na ordem contrária de `ord`. `link(i)` é o maior sufixo com um conjunto `endpos` diferente.

Encontrando a maior substring comum. Construa o automato de sufixos de uma das strings. Itere pela segunda string mantendo o maior sufixo dela que aparece na primeira string. Quando uma transição falha, basta subir por suffix links, e o novo maior tamanho do sufixo vira `len(v)`.

Geometria

Transformações da distância Manhattan. Em 2D,

$$L_1 = \max(|\Delta_+|, |\Delta_-|)$$

$$\Delta_\pm = (x \pm y)' - (x \pm y)$$

Na prática:

$$\max L_1 = \max(D_+, D_-)$$

$$D_\pm = \max(x \pm y) - \min(x \pm y)$$

Teorema de Euler para grafos planares. Num grafo planar com v vértices, e arestas, f faces e k componentes conexas,

$$v - e + f = 1 + k$$

Teorema de Pick. Para um polígono simples na grid,

$$A = I + \frac{B}{2} - 1,$$

onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.

Pontos de grid em um segmento. O número de pontos de grid no segmento de (x_1, y_1) até (x_2, y_2) é

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1.$$

Fórmula de Heron. Para lados a, b, c e semiperímetro

$$s = \frac{a+b+c}{2},$$

a área do triângulo é

$$\sqrt{s(s-a)(s-b)(s-c)}.$$

Raio da circunferência circunscrita e inscrita. Para área do triângulo Δ:

$$R = \frac{abc}{4\Delta}, \qquad r = \frac{\Delta}{s}$$

onde $s = \frac{a+b+c}{2}$.

Teorema de Helly em $\mathbb{R}^d$. Para conjuntos convexos em $\mathbb{R}^d$, se todo subconjunto de $d + 1$ deles intersecta, então todos intersectam.

DP

Formular DP Se um problema pode ser resolvido com DP, mesmo que a solução óbvia resultaria em TLE, escreva a formulação da DP. Talvez seja possível otimizar a DP e resolver rápido o suficiente, ou mesmo generalizar a forma obtida para algo mais simples/fácil de calcular.

Otimizar com matrizes Especialmente útil quando o problema pode ser resolvido com uma DP que depende apenas

de uma janela de estados anteriores de tamanho fixo. Tentar representar as transições da DP como multiplicação de matrizes.

Otimizar com funções Pode ser que a DP tome o formato de uma função. Neste caso, talvez seja possível calcular o valor da DP em outros pontos descobrindo a forma da função, como usando interpolação ou outra forma.

DP contando subsequências `DP[i]` = número de subsequências distintas até a posição `i`. Temos que:

$$DP[i] = 2 * DP[i-1] - DP[j-1]$$

onde `j` é a última posição que o valor na posição `i` apareceu.

Convolução Min-Plus. A convolução Min-Plus de duas sequências convexas pode ser calculada com greedy nas sequências de diferença. A convolução Min-Plus de duas sequências convexas também é convexa.

Tabela de funções geradoras

$[x^n]F(x)$	$F(x)$
1	$\frac{1}{1-x}$
a^n	$\frac{1}{1-ax}$
n	$\frac{x}{(1-x)^2}$
$\binom{n+k}{k}$	$\frac{1}{(1-x)^{k+1}}$
$\binom{n-a+k-1}{k-1}$	$\frac{x^a}{(1-x)^k}$
$\binom{k}{n}$	$(1+x)^k$
C_n	$\frac{1-\sqrt{1-4x}}{2x}$
F_n	$\frac{x}{1-x-x^2}$
$\frac{D_n}{n!}$	$\frac{e^{-x}}{1-x}$
$\frac{1}{n}$	$\log\frac{1}{1-x}$

Debugging

Wrong Answer

- reler o enunciado, os exemplos e as restrições.
- sua solução passa os exemplos?
- a resposta pede valor exato ou módulo?
- o módulo digitado está correto, e é o mesmo que o enunciado?
- verificar todas as operações aritméticas consideram o módulo, buscar todas as operações no código e verificar manualmente.
- verificar casos especiais, $n = 0$, $n = 1$, etc.
- testar manualmente todos os casos com input pequeno.
- considerou que $-x \mod y$ pode ser negativo?
- ver manualmente todos os índices de array e verificar se é possível que o índice esteja fora do array.
- limpou todas as variáveis globais em casos de teste?
- todas as variáveis são inicializadas corretamente?
- arrays são inicializados antes de serem usados?
- divisão por 0 pode causar `NaN`

Runtime Error

- ver manualmente todos os índices de array e verificar se é possível que o índice esteja fora do array.
- os índices vão até $n-1$ ou até n? o array tem o tamanho certo? ($n+1$ se os índices vão até n)
- os limites dos loops estão corretos? trocou m e n?
- os arrays tem os tamanhos corretos? o tamanho era $2n$ em vez de n?
- divisão por 0?

Time Limit Exceeded

- algum loop infinito acidental?
- recursão infinita? impediu de visitar o mesmo estado várias vezes?
- algum fator de log extra? talvez por causa de um set ou map.
- a complexidade amortizada está realmente correta?
- sem querer copiou objetos grandes várias vezes?
- inicializou/limpou objetos grandes sempre em cada caso de teste?

Implementações

- self-loops e multi-edges importam no grafo?
- tratou o caso desconexo?
- quando buscar arestas, `(a, b)` e `(b, a)` ambas encontram a mesma aresta?
- ao processar intervalos ativos e inativos, o que acontece com os intervalos ativos quando chegam ao final do processamento? Ficam inativos?